Tech Stack and Judge Q&A

PPE Compliance Detection • read this, then practise the answers out loud

Part 1 — What we used (short)

Tool	What it is and what we used it for
Python 3.12	The one language for all our code: detection, matching logic, rules, app, tests, training scripts.
Ultralytics YOLO11	The AI model that finds persons, helmets and vests. We fine-tuned a pretrained version.
PyTorch (CUDA)	The engine under YOLO. It ran training on the laptop's NVIDIA RTX 5060 GPU.
OpenCV	Reads images and video, draws the boxes and labels, brightens dark photos.
Streamlit	Builds the web app (upload a photo, see the result, download files).
pandas, NumPy	Tables, CSV/JSON export, image arrays.
pytest	Runs our 69 automatic tests.
HTML/CSS, PowerShell	Only for the batch report page and PDFs, and for typing run commands on Windows.
Datasets	Construction-PPE (Ultralytics, AGPL-3.0) and CHVG (Ferdous & Ahsan 2022, CC BY 4.0), both public and credited.
Claude Code	AI coding assistant. We directed the design and reviewed and tested the work. It is credited in the README.

> **Who wrote what — the honest answer.** The design and logic are ours: the zones, the one-to-one matching, the three outcomes, the rules. We built the code with an AI coding assistant, Claude, then reviewed it, tested it on our own photos, and can explain every module. The YOLO detection model and the image and web libraries are open-source libraries, not our code.

Part 2 — Questions a judge may ask

A. The problem and the idea

What problem does this solve?	Supervisors can't watch every worker in a busy area, so people without helmets or vests get missed. We check every worker automatically and flag who is missing what.
What is the input and the output?	Input: a photo or short video. Output: the image with a green, red or amber verdict per worker, a table, a violation summary, and CSV/JSON files.
What is new or different about your approach?	We don't just detect a helmet; we check it is on *that worker's* head, and each helmet or vest can belong to only one worker. A helmet elsewhere in the picture can't clear everyone.
Who would use this?	Safety supervisors on construction sites and factories, as an assistant that points them at likely violations. A human still reviews the amber cases.

B. How it works

Walk us through the pipeline.	Brighten if dark, detect persons/helmets/vests, match each item to a worker by body zone, apply the rule, draw the verdict, summarise. Only the detection step uses the AI model; the rest is logic we wrote.
How do you know a helmet belongs to this worker?	Each worker has a head zone, the top 35% of their box. A helmet counts only if its centre is in that zone and most of it overlaps it. Vests must be on the torso and large enough.
What if a helmet is in a hand or on a table?	It is outside every head zone, so nobody gets credit. The worker is flagged and the helmet is shown as "unassigned". Unit tests cover this case.
What does NEEDS_REVIEW mean?	The system can't judge fairly: the worker is tiny, cut off, crowded, or the helmet was only seen with low confidence. We ask a human instead of accusing someone wrongly.
Why don't you train a "no helmet" class?	A bare head is varied and hard to detect reliably. It is more robust to find what is there and infer that something is missing when it isn't matched to the worker.
How do you handle many workers, occlusion, distance and poor light?	Each worker is numbered and judged separately. Occluded or crowded workers become REVIEW. For distance we use a larger input size and zoomed tiles. Dark photos are brightened automatically.

C. Data and training

Where did your data come from?	Two public datasets, both credited: Construction-PPE (1,416 photos) and CHVG (1,699 photos). We did not collect them ourselves.
Did you train from scratch?	No. We fine-tuned a pretrained YOLO11s. It already knew objects and people, so we only taught it three classes.
How was training so quick?	Transfer learning plus a laptop GPU. Round 1 took 36 minutes and round 2 about 40.
Why only 3,000 or so images? Isn't that small?	It is small, and we say so. Pretrained models need far less data. More varied data is our top next step, especially hard cases.
Why did you train a second round?	Round 1 scored well on photos like its training photos (0.866) but only 0.758 on photos from other sites. A second dataset with different sites and four helmet colours was meant to close that gap.

D. Accuracy and testing

What is your accuracy?	Round 1: 0.866 mAP@50 on the original test photos and 0.758 on photos from other sites. Round 2: 0.829 and 0.882, but round 2 saw part of the second collection, so that last number isn't a fair comparison. We call the other-sites number the more realistic one.
What does mAP@50 mean?	A score from 0 to 1 that combines how many real objects we find and how accurate the boxes are. Higher is better.
How did you test the matching logic?	69 automatic tests with hand-made boxes (helmet in hand, helmet on a table, one helmet between two workers, and more), plus visual checks on real site photos.

Did you test the hard cases?	We checked them on a few real photos and in unit tests. We have **not** yet measured them on a full test set of our own; the scoring script is built and waiting for photos and an answer key.
Could your numbers be inflated?	Yes. The datasets have many near-identical frames, so the original-split score flatters the model, and any thresholds tuned on a small set will look optimistic. That is why we also report the other-sites score.

E. Limits and honesty

Where does it fail?	Small or background workers can be missed, life jackets or hi-vis shirts can look like vests, seated or bending workers may become REVIEW, and two close workers can merge into one box.
What about false alarms?	Our design leans cautious: unsure cases become REVIEW rather than violations, and every verdict comes with a plain-English reason so a human can check it.
Is it real-time?	We have not benchmarked real-time speed. For video we analyse every second frame and skip the slower zoomed-tile mode. Real-time on many cameras would need further work.
Does it work on video?	The code and its tests exist: each worker is tracked and a violation is reported only after it persists for 5 of the last 8 usable frames. We have tested it on synthetic video, not yet on a real clip.
Why those thresholds (0.35, 0.30, 35% head zone)?	They are sensible starting values, collected in one config file. They have been lightly checked on real photos and are meant to be tuned on a larger test set.

F. Choices, ethics and next steps

Why YOLO and Python?	YOLO is fast, widely used and runs on a laptop; Python is the standard language for this. We did not benchmark other model families.
What about privacy?	The system doesn't identify anyone; it only numbers workers in an image. A real deployment would still need worker consent, a clear policy and secure storage of footage.
Could it check other equipment, like goggles?	The required items are configurable, but a new item needs training data and a body zone definition. Helmet and vest are what we built and tested.
What would you do with more time?	Collect and label our own hard-case photos, add hard-negative images such as hi-vis non-vests and life jackets, tune thresholds on that set, test on real video, and benchmark speed.
What licences apply?	Ultralytics YOLO and the Construction-PPE dataset are AGPL-3.0, so we publish our source; CHVG is CC BY 4.0, credited in the README.

Part 3 — Tricky questions and safe replies

If a judge says…	Say…
“So the AI did all the work?”	“We designed the approach and the logic, directed an AI coding assistant, and then reviewed, tested and measured it ourselves. The detection model is an open-source one we fine-tuned. We credit all of that in the README.”
“Is 0.88 your real accuracy?”	“That number is on photos from a collection the model partly trained on. On other sites, round 1 scored 0.76, so we treat our true performance as somewhere around that and we’re upfront about it.”
“Show me it failing.”	Have one failure photo ready (for example the rafting life-jackets photo or a missed background worker) and explain why it fails and how you would fix it. Choosing your own failure shows you understand the system.
“What if I ask about a part you didn’t work on?”	“[Name] handled that part, I’ll let them answer. From my side, I can explain how the whole pipeline fits together.” Then give the six-step pipeline.
“Why should we trust it on a real site?”	“You shouldn’t yet without testing on that site’s photos. That is why it hands unsure cases to a person and explains every decision. It is an assistant for supervisors, not a replacement.”

If you don’t know an answer: say so plainly and explain what you do know. “We haven’t measured that yet, but here is how we would” is a strong answer. A confident wrong answer is the weakest one.

Never say: “we trained it from scratch” • “we collected the data” • “it is 100% accurate” • “it handles every hard case” • “we wrote every line ourselves”.